POC-03 Real-Time Data Platform

Event Hubs, Databricks, Fabric & Synapse

Beginner Implementation, Troubleshooting & Verification Guide

Contents

What This POC Builds

A near-real-time shipment analytics platform where Python generates shipment events, Azure Event Hubs receives them, Databricks processes them into Bronze/Silver/Gold layers, and downstream analytics can be served with SQL or real-time KQL patterns.

Validated path

Event Hubs producer/receiver worked. Databricks Serverless ingested 50 raw events into a Unity Catalog Volume. The repaired Silver layer contained 48 clean unique events with 0 duplicate event IDs. Gold KPIs were exported and Databricks SQL views were created.

Reference-only path

Fabric Eventstream/Eventhouse/KQL, Synapse Serverless SQL, Purview, and semantic modeling remain implementation references unless you complete those steps separately. The final working Gold data in this run was stored under a Databricks /Volumes path.

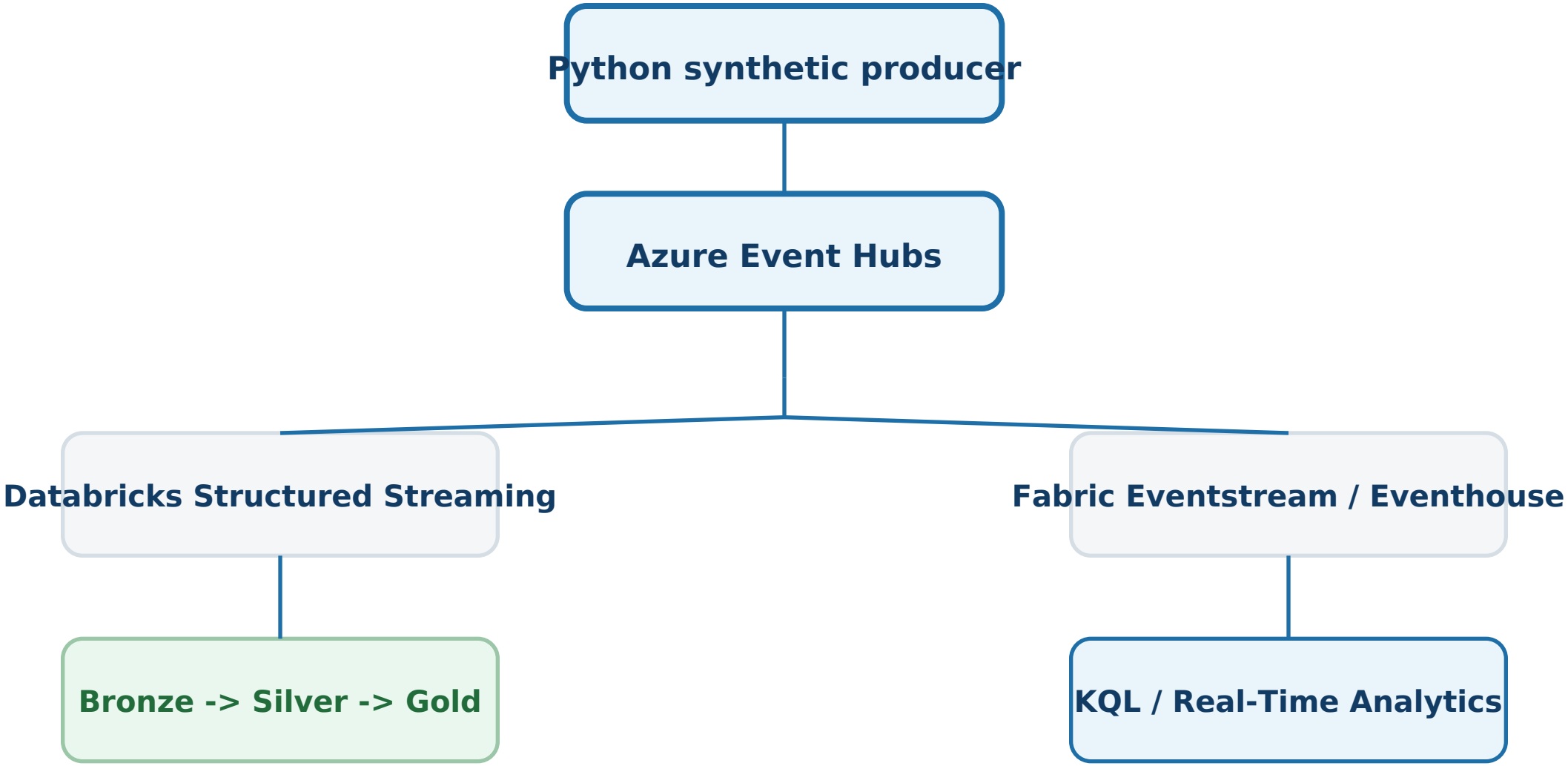
Validated Components - Part 1

Component	Status	Result
Resource group	Validated	Created successfully
Event Hubs namespace	Validated	Producer/receiver proved connectivity
Storage account	Validated	ADLS Gen2 account created
Local producer	Validated	Initial error fixed; sends succeeded
Local receiver	Validated	Messages read from Event Hubs
Databricks workspace	Validated	Workspace deployed

Validated Components - Part 2

Component	Status	Result
Bronze ingestion	Validated	50 raw events written
Unity Catalog Volume	Validated	Delta files/logs visible
Silver layer	Validated after recovery	48 clean rows; 0 duplicate IDs
Gold KPIs	Validated	Parquet summary exported
Databricks SQL	Validated	Views created over Silver and Gold
Fabric / Synapse / Purview	Reference	Not validated in the completed run

Architecture



Event Schema

JSON

```
{
  "event_id": "evt-001",
  "order_id": 1001,
  "event_type": "SHIPPED",
  "event_ts": "2026-08-28T10:00:00Z",
  "region": "IN-SOUTH",
  "revenue": 1299.50,
  "fulfillment_minutes": 120,
  "producer_seq": 1
}
```

Why these fields matter

event_id supports deduplication; event_ts supports event-time processing and watermarks; region supports partitioning/grouping; revenue and fulfillment_minutes make Gold KPIs and semantic measures possible.

Project Structure

```
POC_03_REALTIME_FABRIC_SYNAPSE/
├── .env.example
├── requirements.txt
├── producer/
│   ├── send_events.py
│   └── receive_events.py
├── databricks/
│   ├── stream_to_delta.py
│   ├── inspect_and_validate.py
│   └── export_gold_parquet.py
├── fabric/setup_notes.md
├── kql/
│   ├── create_shipment_table.kql
│   └── shipment_queries.kql
├── synapse/serverless_queries.sql
├── semantic_model/measures.dax
├── governance/purview_notes.md
├── monitoring/monitoring_checklist.md
├── docs/
│   ├── azure_portal_setup.md
│   ├── troubleshooting.md
│   └── cleanup.md
└── evidence/
```

First-Time Local Setup

Run from the project root on Windows Command Prompt or PowerShell.

PowerShell / CMD

```
cd <path-to-project>\POC_03_REALTIME_FABRIC_SYNAPSE
python -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements.txt
copy .env.example .env
```

Alternative environment

If you already use a shared Python environment, activate it instead of creating .venv, then install requirements. The important point is that azure-eventhub and python-dotenv are available.

Command Runbook - Correct Order

1. Connectivity test

```
python producer\send_events.py --count 20 --delay 0.2
```

2. Read messages

```
python producer\receive_events.py --max-events 10 --starting-position -1
```

3. Main Databricks test batch

```
python producer\send_events.py --count 50
```

4. Databricks notebook 1

`stream_to_delta.py`

5. Databricks notebook 2

`inspect_and_validate.py`

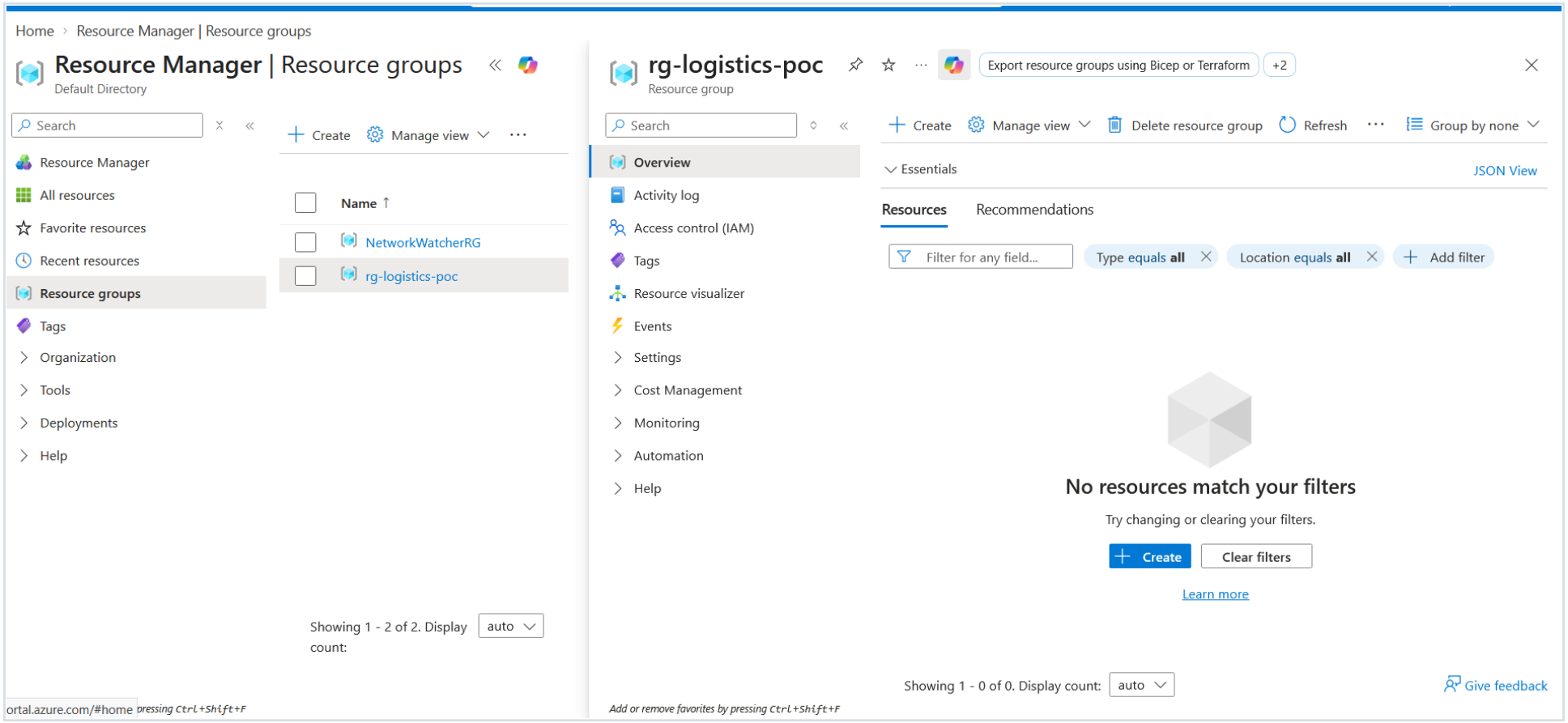
6. Databricks notebook 3

`export_gold_parquet.py`

7. SQL verification

Create/refresh Silver and Gold SQL views, then run verification queries

Create the Resource Group

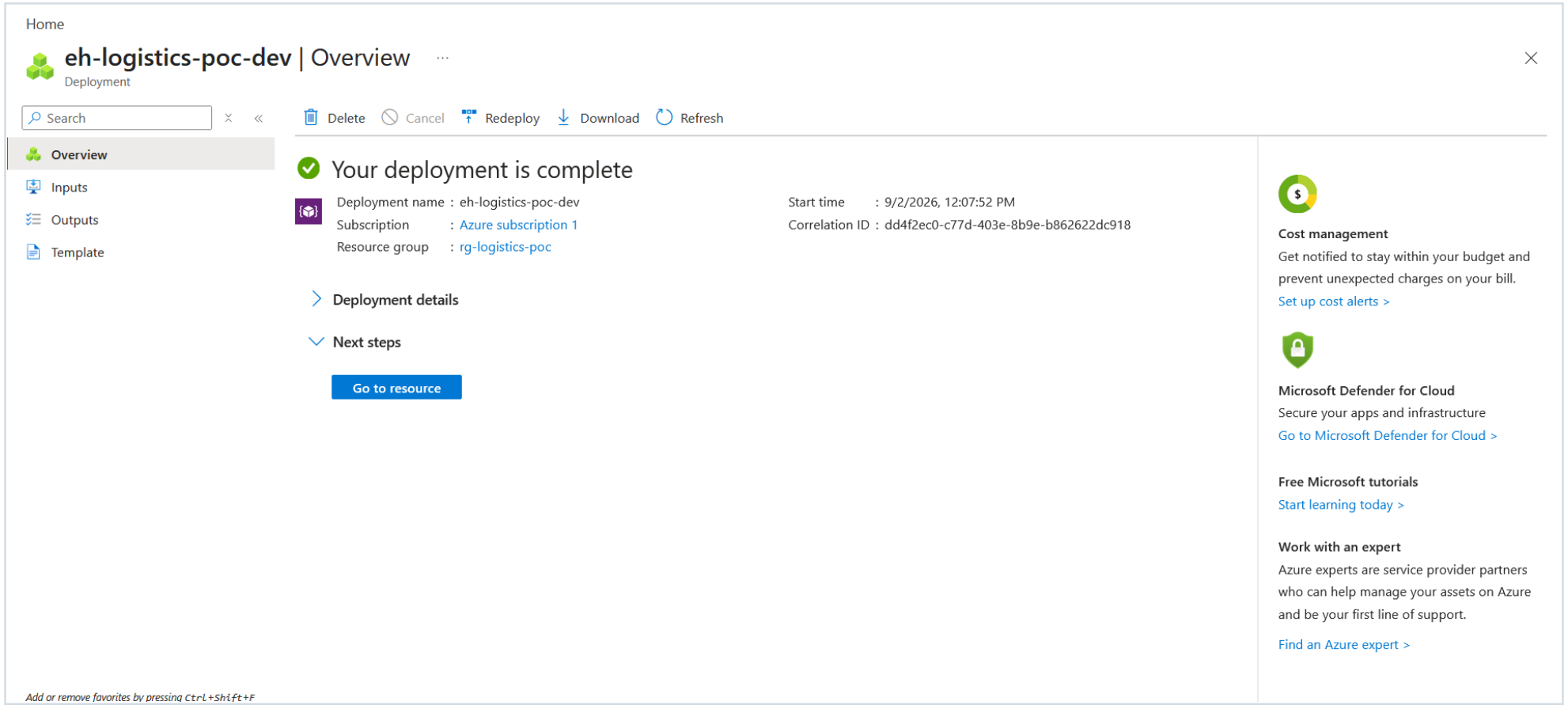


Resource group created successfully

Key point

Keep all POC-only Azure resources in one resource group when practical. This makes cleanup much easier.

Create Azure Event Hubs

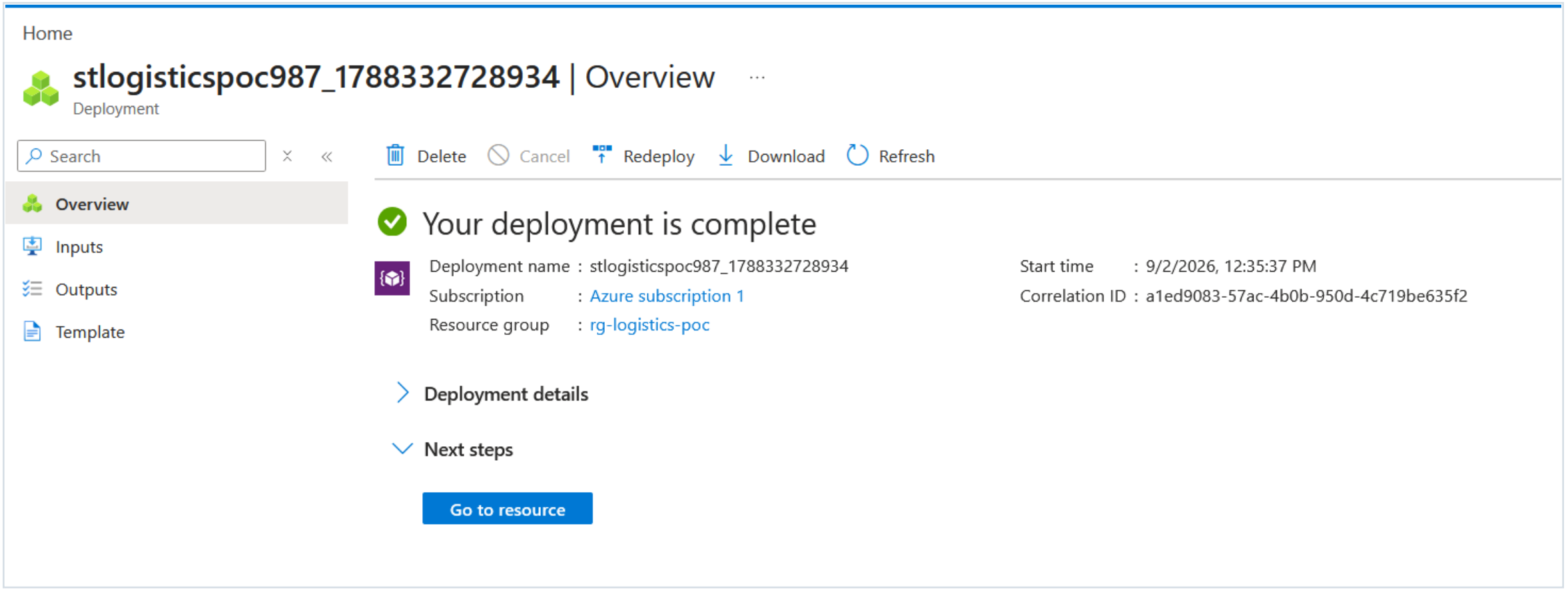


Event Hubs namespace deployment completed

Key point

Create the Event Hub entity shipment-events inside the namespace and use a small/default partition count for this learning POC.

Create ADLS Gen2 Storage

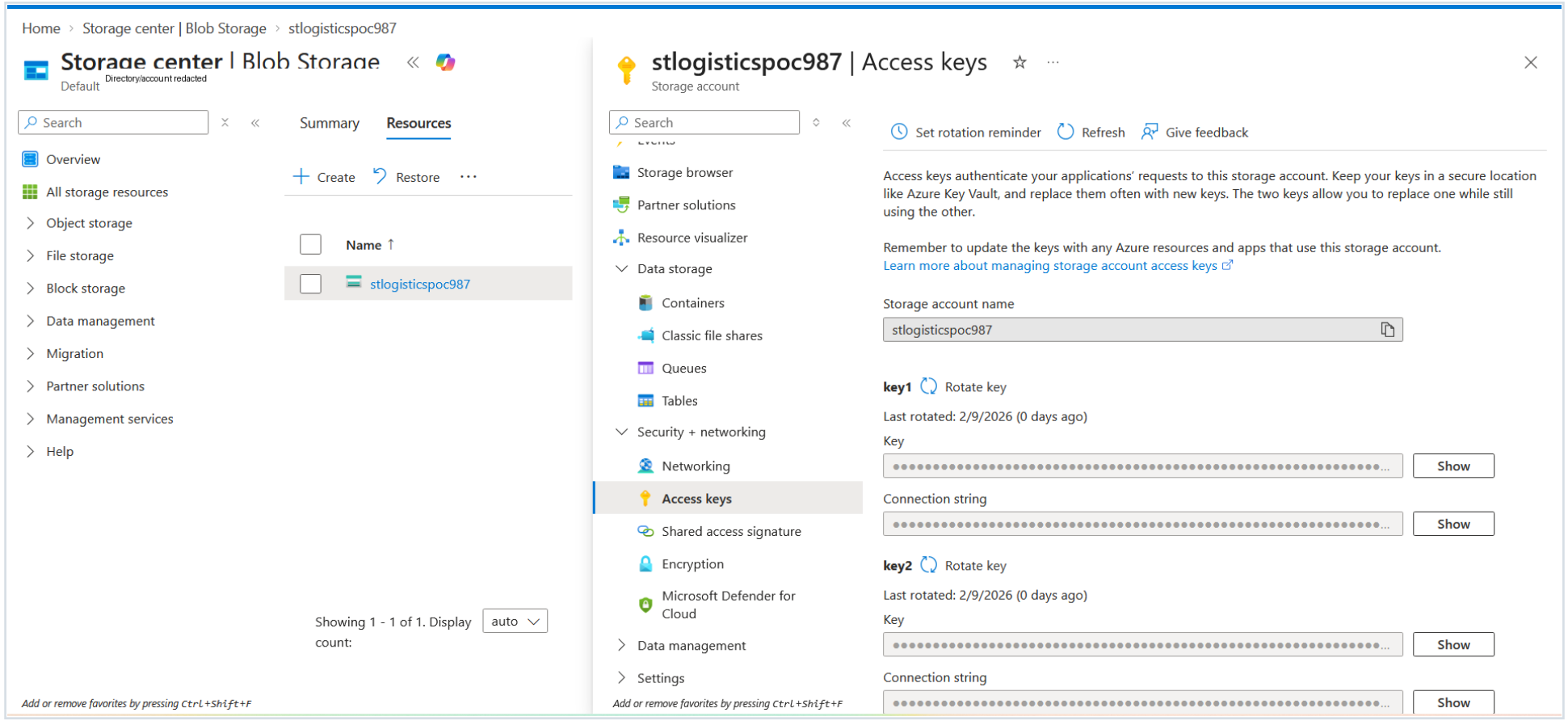


Storage account deployment completed

Key point

Use Standard performance, a low-cost redundancy option, enable Hierarchical namespace, and create a private realtime container.

Storage Access Key - Temporary Debugging Path

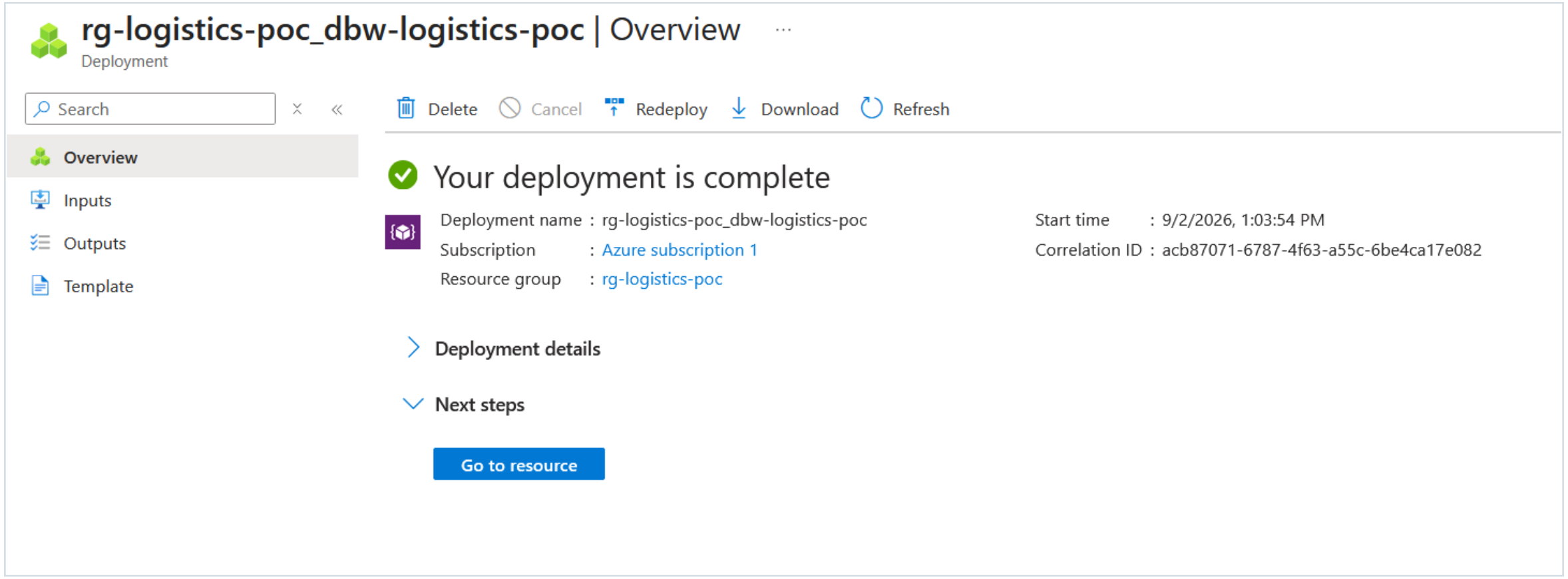


Storage access-key location

Key point

The direct storage-key approach was only a troubleshooting path. Do not hard-code storage keys. Prefer Unity Catalog governed access, managed identity, or Databricks secrets.

Create Azure Databricks



Azure Databricks workspace deployment completed

Key point

The final working path used Databricks Serverless compute and a Unity Catalog managed Volume.

.env Configuration - Values to Collect

Variable	Value / Location
EVENT_HUB_CONNECTION_STRING	Event Hubs policy -> Connection string-primary key
EVENT_HUB_NAME	shipment-events
EVENT_HUB_CONSUMER_GROUP	$Default
EVENT_HUB_NAMESPACE	Namespace name only
EVENT_HUB_KAFKA_BOOTSTRAP	<namespace>.servicebus.windows.net:9093
ADLS_ACCOUNT_NAME	Storage account name
ADLS_CONTAINER_NAME	realtime

.env Safe Template

.env

```
EVENT_HUB_CONNECTION_STRING=Endpoint=sb://<namespace>.servicebus.windows.net/;SharedAccessKeyName=<p
    olicy>;SharedAccessKey=<secret>
EVENT_HUB_NAME=shipment-events
EVENT_HUB_CONSUMER_GROUP=$Default
EVENT_COUNT=200
EVENT_DELAY_SECONDS=0.15
DUPLICATE_EVERY=25
LATE_EVENT_EVERY=40
ADLS_ACCOUNT_NAME=<storage-account-name>
ADLS_CONTAINER_NAME=realtime
DELTA_BRONZE_PATH=abfss://realtime@<storage-account-name>.dfs.core.windows.net/delta/bronze/shipment
    _events
DELTA_SILVER_PATH=abfss://realtime@<storage-account-name>.dfs.core.windows.net/delta/silver/shipment
    _events
GOLD_PARQUET_PATH=abfss://realtime@<storage-account-name>.dfs.core.windows.net/gold/shipment_summary
EVENT_HUB_NAMESPACE=<namespace>
EVENT_HUB_KAFKA_BOOTSTRAP=<namespace>.servicebus.windows.net:9093
```

Issue 1 - Producer Connection String Error

```
(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse>python producer/send_events.py
Traceback (most recent call last):
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse\producer\send_events.py", line 141, in <module>
    main()
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse\producer\send_events.py", line 137, in main
    send_events(args.count, args.delay, args.duplicate_every, args.late_every)
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse\producer\send_events.py", line 70, in send_events
    producer = EventHubProducerClient.from_connection_string(
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\ermdi\projects\common_venv_py312\.venv\Lib\site-packages\azure\eventhub\_producer_client.py", line 545, in from_connection_string
    constructor_args = cls._from_connection_string(
                       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\ermdi\projects\common_venv_py312\.venv\Lib\site-packages\azure\eventhub\_client_base.py", line 329, in _from_connection_string
    host, policy, key, entity, token, token_expiry, emulator = _parse_conn_str(conn_str, **kwargs)
                                                               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\ermdi\projects\common_venv_py312\.venv\Lib\site-packages\azure\eventhub\_client_base.py", line 88, in _parse_conn_str
    conn_settings = core_parse_connection_string(conn_str, case_sensitive_keys=check_case)
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\ermdi\projects\common_venv_py312\.venv\Lib\site-packages\azure\core\utils\_connection_string_parser.py", line 27, in parse_connection_strin
g
    raise ValueError("Connection string is either blank or malformed.")
ValueError: Connection string is either blank or malformed.

(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse>
```

ValueError: connection string is blank or malformed

Key point

This usually means the wrong Azure field was copied, the .env file was not loaded, the filename is wrong, or the connection string formatting is invalid.

Fix Issue 1 - Validate .env Before Retrying

1. Copy Connection string-primary key, not only Primary key.
2. Confirm the filename is exactly .env, not .env.txt.
3. Do not add spaces around the variable name or comment out the line.
4. Confirm the value begins with Endpoint=sb://.
5. Check what Python actually reads, then rerun the producer.

Diagnosis and retry

```
Get-ChildItem -Hidden .env*
python -c "import os; from dotenv import load_dotenv; load_dotenv(); print('Length:', len(os.getenv(
    'EVENT_HUB_CONNECTION_STRING') or ''))"
python producer\send_events.py
```

Producer Verification - Success

```
[168/200] NORMAL    id=evt-9a739fb158ce type=SHIPPED    region=IN-WEST ts=2026-09-02T07:19:04Z
[169/200] NORMAL    id=evt-4389556967d2 type=PACKED     region=IN-EAST ts=2026-09-02T07:19:05Z
[170/200] NORMAL    id=evt-a9576821da96 type=DELAYED    region=IN-WEST ts=2026-09-02T07:19:05Z
[171/200] NORMAL    id=evt-037c0b970c9b type=SHIPPED    region=IN-NORTH ts=2026-09-02T07:19:06Z
[172/200] NORMAL    id=evt-7eb72f74fee9 type=PACKED     region=IN-EAST ts=2026-09-02T07:19:06Z
[173/200] NORMAL    id=evt-a48492969b00 type=CREATED    region=IN-WEST ts=2026-09-02T07:19:06Z
[174/200] NORMAL    id=evt-afc9f3a5cbc7 type=DELIVERED  region=IN-EAST ts=2026-09-02T07:19:07Z
[175/200] DUPLICATE id=evt-afc9f3a5cbc7 type=DELIVERED  region=IN-EAST ts=2026-09-02T07:19:07Z
[176/200] NORMAL    id=evt-b1494312d311 type=SHIPPED    region=IN-SOUTH ts=2026-09-02T07:19:07Z
[177/200] NORMAL    id=evt-0547d6932fb3 type=DELAYED    region=IN-NORTH ts=2026-09-02T07:19:08Z
[178/200] NORMAL    id=evt-83806a05ec90 type=CREATED    region=IN-SOUTH ts=2026-09-02T07:19:08Z
[179/200] NORMAL    id=evt-577d1f2ad115 type=DELIVERED  region=IN-EAST ts=2026-09-02T07:19:09Z
[180/200] NORMAL    id=evt-a17fa52f752e type=PACKED     region=IN-SOUTH ts=2026-09-02T07:19:09Z
[181/200] NORMAL    id=evt-369c9c532d4f type=IN_TRANSIT region=IN-WEST ts=2026-09-02T07:19:09Z
[182/200] NORMAL    id=evt-08bcf099bbbf type=IN_TRANSIT region=IN-NORTH ts=2026-09-02T07:19:10Z
[183/200] NORMAL    id=evt-6f04c50ba215 type=PACKED     region=IN-SOUTH ts=2026-09-02T07:19:10Z
[184/200] NORMAL    id=evt-1d20a96e32af type=PACKED     region=IN-EAST ts=2026-09-02T07:19:11Z
[185/200] NORMAL    id=evt-95975e9ad9bb type=DELAYED    region=IN-WEST ts=2026-09-02T07:19:11Z
[186/200] NORMAL    id=evt-a3309418b466 type=PACKED     region=IN-NORTH ts=2026-09-02T07:19:11Z
[187/200] NORMAL    id=evt-633a01df7738 type=DELIVERED  region=IN-NORTH ts=2026-09-02T07:19:12Z
[188/200] NORMAL    id=evt-ee02e20ac5b0 type=PACKED     region=IN-EAST ts=2026-09-02T07:19:12Z
[189/200] NORMAL    id=evt-a5161ac6a187 type=SHIPPED    region=IN-SOUTH ts=2026-09-02T07:19:13Z
[190/200] NORMAL    id=evt-184820e1a25f type=DELIVERED  region=IN-WEST ts=2026-09-02T07:19:13Z
[191/200] NORMAL    id=evt-8073c63e6ecd type=SHIPPED    region=IN-NORTH ts=2026-09-02T07:19:13Z
[192/200] NORMAL    id=evt-d267c573fea8 type=SHIPPED    region=IN-WEST ts=2026-09-02T07:19:14Z
[193/200] NORMAL    id=evt-93f37ab95dad type=PACKED     region=IN-SOUTH ts=2026-09-02T07:19:14Z
[194/200] NORMAL    id=evt-000b2a08bd99 type=CREATED    region=IN-WEST ts=2026-09-02T07:19:15Z
[195/200] NORMAL    id=evt-0264ddf8604e type=DELIVERED  region=IN-SOUTH ts=2026-09-02T07:19:15Z
[196/200] NORMAL    id=evt-c264682cdff3 type=SHIPPED    region=IN-EAST ts=2026-09-02T07:19:15Z
[197/200] NORMAL    id=evt-4ed1aa278289 type=DELIVERED  region=IN-WEST ts=2026-09-02T07:19:16Z
[198/200] NORMAL    id=evt-5ecaabd51a4e type=SHIPPED    region=IN-SOUTH ts=2026-09-02T07:19:16Z
[199/200] NORMAL    id=evt-94918d3c3d33 type=PACKED     region=IN-EAST ts=2026-09-02T07:19:17Z
[200/200] DUPLICATE id=evt-94918d3c3d33 type=PACKED     region=IN-EAST ts=2026-09-02T07:19:17Z

Summary
  sent attempts : 200
  duplicates    : 8
  late events   : 4
Check Event Hubs > Monitoring > Metrics > Incoming Messages.

(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse>
```

Event producer succeeded after .env correction

Key point

The successful run sent 200 events. For later Databricks available-now testing, a smaller 50-event batch was used.

Event Hubs Receiver Verification

```
(common-venv-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\poc_03_realtime_fabric_synapse>python producer/receive_events.py
Receiving up to 20 events from shipment-events (consumer group=$Default, starting_position=-1)
[001] partition=0 offset=0 sequence=0 payload={'event_id': 'evt-8ef4eaaec6ff', 'order_id': 1003, 'event_type': 'CREATED', 'event_ts': '2026-09-02T07:17:58Z'
, 'region': 'IN-WEST', 'revenue': 2977.2, 'fulfillment_minutes': 720, 'producer_seq': 3}
[002] partition=0 offset=296 sequence=1 payload={'event_id': 'evt-45efe6cf89bd', 'order_id': 1004, 'event_type': 'PACKED', 'event_ts': '2026-09-02T07:17:59Z
', 'region': 'IN-WEST', 'revenue': 774.51, 'fulfillment_minutes': 281, 'producer_seq': 4}
[003] partition=0 offset=592 sequence=2 payload={'event_id': 'evt-3fca5b1a4b08', 'order_id': 1006, 'event_type': 'DELIVERED', 'event_ts': '2026-09-02T07:18:
00Z', 'region': 'IN-WEST', 'revenue': 1106.75, 'fulfillment_minutes': 463, 'producer_seq': 6}
[004] partition=1 offset=0 sequence=0 payload={'event_id': 'evt-2ca2fc7fc93b', 'order_id': 1001, 'event_type': 'SHIPPED', 'event_ts': '2026-09-02T07:17:55Z'
, 'region': 'IN-EAST', 'revenue': 300.13, 'fulfillment_minutes': 304, 'producer_seq': 1}
[005] partition=0 offset=896 sequence=3 payload={'event_id': 'evt-96231f2a0c62', 'order_id': 1008, 'event_type': 'SHIPPED', 'event_ts': '2026-09-02T07:18:01
Z', 'region': 'IN-WEST', 'revenue': 571.77, 'fulfillment_minutes': 436, 'producer_seq': 8}
[006] partition=1 offset=296 sequence=1 payload={'event_id': 'evt-464de99e4d1e', 'order_id': 1002, 'event_type': 'DELIVERED', 'event_ts': '2026-09-02T07:17:
58Z', 'region': 'IN-SOUTH', 'revenue': 2189.46, 'fulfillment_minutes': 349, 'producer_seq': 2}
[007] partition=0 offset=1192 sequence=4 payload={'event_id': 'evt-d10e5fda2a7c', 'order_id': 1010, 'event_type': 'CREATED', 'event_ts': '2026-09-02T07:18:0
1Z', 'region': 'IN-NORTH', 'revenue': 3766.99, 'fulfillment_minutes': 557, 'producer_seq': 10}
[008] partition=1 offset=600 sequence=2 payload={'event_id': 'evt-b34bdcd0a43c', 'order_id': 1005, 'event_type': 'CREATED', 'event_ts': '2026-09-02T07:17:59
Z', 'region': 'IN-SOUTH', 'revenue': 3365.32, 'fulfillment_minutes': 183, 'producer_seq': 5}
[009] partition=0 offset=1496 sequence=5 payload={'event_id': 'evt-3b2a6e307127', 'order_id': 1012, 'event_type': 'IN_TRANSIT', 'event_ts': '2026-09-02T07:1
8:02Z', 'region': 'IN-WEST', 'revenue': 651.76, 'fulfillment_minutes': 609, 'producer_seq': 12}
[010] partition=1 offset=904 sequence=3 payload={'event_id': 'evt-fcf614256ea0', 'order_id': 1007, 'event_type': 'PACKED', 'event_ts': '2026-09-02T07:18:00Z
', 'region': 'IN-SOUTH', 'revenue': 3497.85, 'fulfillment_minutes': 591, 'producer_seq': 7}
[011] partition=0 offset=1800 sequence=6 payload={'event_id': 'evt-aa9f7787cdeb', 'order_id': 1014, 'event_type': 'DELIVERED', 'event_ts': '2026-09-02T07:18
:03Z', 'region': 'IN-WEST', 'revenue': 539.8, 'fulfillment_minutes': 707, 'producer_seq': 14}
[012] partition=1 offset=1208 sequence=4 payload={'event_id': 'evt-03c641182a57', 'order_id': 1009, 'event_type': 'SHIPPED', 'event_ts': '2026-09-02T07:18:0
1Z', 'region': 'IN-SOUTH', 'revenue': 290.44, 'fulfillment_minutes': 272, 'producer_seq': 9}
[013] partition=0 offset=2104 sequence=7 payload={'event_id': 'evt-011f92bc7c5f', 'order_id': 1015, 'event_type': 'PACKED', 'event_ts': '2026-09-02T07:18:03
Z', 'region': 'IN-WEST', 'revenue': 307.1, 'fulfillment_minutes': 495, 'producer_seq': 15}
[014] partition=1 offset=1512 sequence=5 payload={'event_id': 'evt-ff3b71f750a7', 'order_id': 1011, 'event_type': 'SHIPPED', 'event_ts': '2026-09-02T07:18:0
2Z', 'region': 'IN-SOUTH', 'revenue': 4907.92, 'fulfillment_minutes': 433, 'producer_seq': 11}
[015] partition=0 offset=2400 sequence=8 payload={'event_id': 'evt-a0b30359711f', 'order_id': 1016, 'event_type': 'DELIVERED', 'event_ts': '2026-09-02T07:18
:04Z', 'region': 'IN-WEST', 'revenue': 625.73, 'fulfillment_minutes': 406, 'producer_seq': 16}
[016] partition=1 offset=1816 sequence=6 payload={'event_id': 'evt-2a3597cbc284', 'order_id': 1013, 'event_type': 'SHIPPED', 'event_ts': '2026-09-02T07:18:0
3Z', 'region': 'IN-SOUTH', 'revenue': 697.03, 'fulfillment_minutes': 711, 'producer_seq': 13}
[017] partition=0 offset=2704 sequence=9 payload={'event_id': 'evt-66ac2fdad98b', 'order_id': 1018, 'event_type': 'IN_TRANSIT', 'event_ts': '2026-09-02T07:1
8:05Z', 'region': 'IN-WEST', 'revenue': 4158.48, 'fulfillment_minutes': 573, 'producer_seq': 18}
[018] partition=1 offset=2120 sequence=7 payload={'event_id': 'evt-703eb2169fe2', 'order_id': 1017, 'event_type': 'PACKED', 'event_ts': '2026-09-02T07:18:04
Z', 'region': 'IN-SOUTH', 'revenue': 2579.07, 'fulfillment_minutes': 279, 'producer_seq': 17}
[019] partition=0 offset=3008 sequence=10 payload={'event_id': 'evt-be2bf56fe50f', 'order_id': 1021, 'event_type': 'CREATED', 'event_ts': '2026-09-02T07:18:
06Z', 'region': 'IN-NORTH', 'revenue': 265.54, 'fulfillment_minutes': 202, 'producer_seq': 21}
[020] partition=1 offset=2424 sequence=8 payload={'event_id': 'evt-5c2c990abc2e', 'order_id': 1019, 'event_type': 'SHIPPED', 'event_ts': '2026-09-02T07:18:0
```

Local receiver reads Event Hub messages

Key point

Do not continue to Spark troubleshooting until both local send and receive work. This isolates Event Hubs connectivity from storage or Databricks problems.

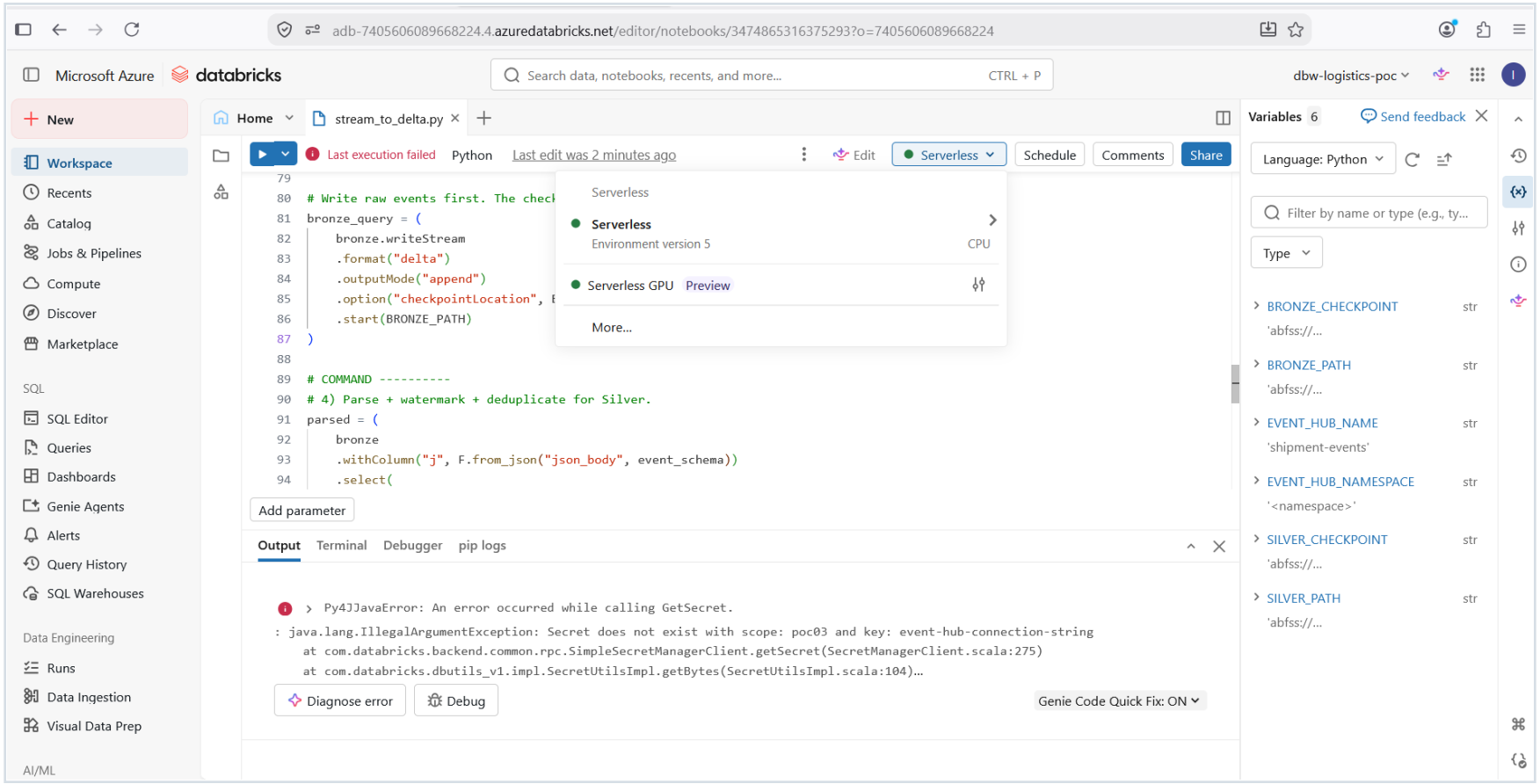
Databricks Serverless - Start Here

1. Open the Databricks workspace and import databricks/stream_to_delta.py.
2. Attach or select Serverless compute.
3. Replace placeholder namespace/path values before the first run.
4. Keep the Event Hubs credential in a Databricks secret when possible.
5. For the final successful storage path, use a Unity Catalog managed Volume rather than spark.conf.set with an account key.

Execution model

On Serverless interactive compute, the successful pattern used trigger(availableNow=True): send a batch first, run the notebook, drain the available events, write Delta, and terminate.

Issue 2 - Databricks Secret Missing



Secret scope/key did not exist

Key point

The notebook attempted dbutils.secrets.get(scope="poc03", key="event-hub-connection-string") before that secret had been created.

Fix Issue 2 - Credential Options

Preferred

Create the Databricks secret scope/key and keep the Event Hubs connection string out of notebook source.

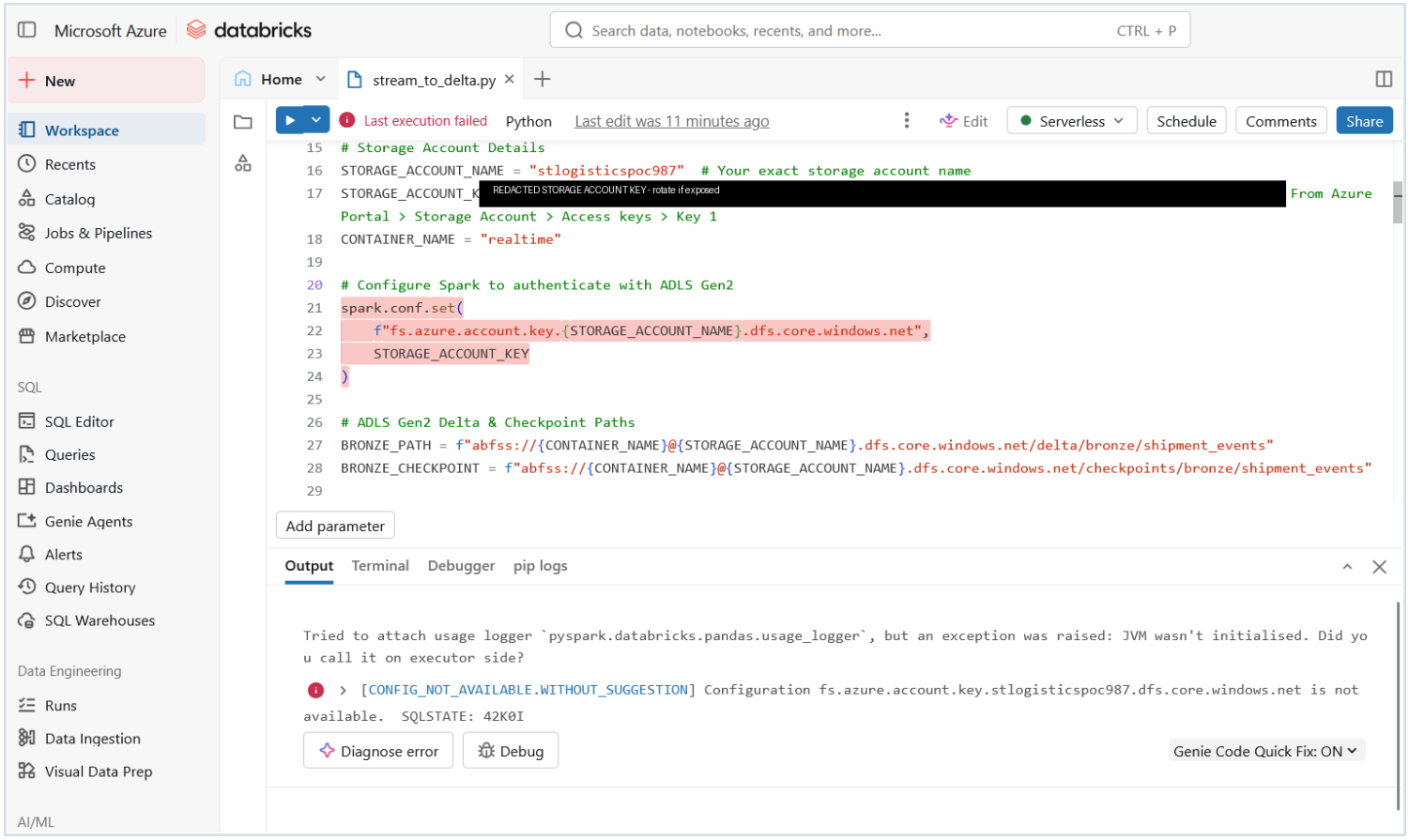
Temporary debugging fallback

A direct connection string can be used only to unblock a private POC troubleshooting session. Never commit or share the notebook, then remove the credential and rotate it if exposed.

Preferred notebook pattern

```
EVENT_HUB_CONNECTION_STRING = dbutils.secrets.get(
    scope="poc03",
    key="event-hub-connection-string"
)
```

Issue 3 - Serverless Storage-Key Configuration Blocked



CONFIG_NOT_AVAILABLE for fs.azure.account.key

Key point

The visible JVM warning was secondary. The primary problem was that Serverless did not allow this Spark storage-key configuration.

Fix Issue 3 - Use a Unity Catalog Volume

Three approaches were considered: switch to a single-user all-purpose cluster for direct key configuration, use Unity Catalog external storage credentials for ADLS, or use a managed Unity Catalog Volume. The successful POC path used the managed Volume.

Final Serverless-compatible storage paths

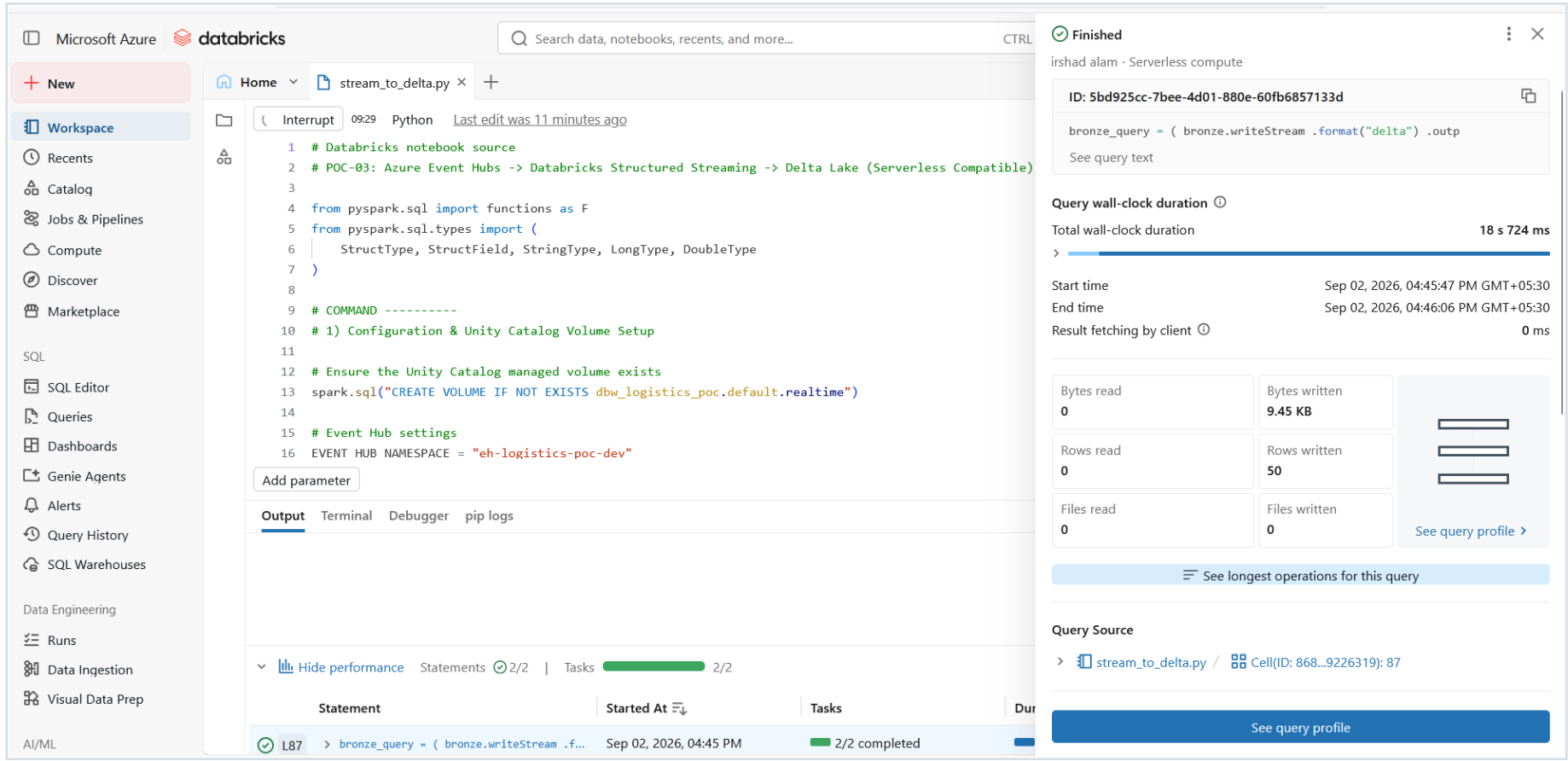
```
spark.sql("CREATE VOLUME IF NOT EXISTS dbw_logistics_poc.default.realtime")
VOLUME_BASE = "/Volumes/dbw_logistics_poc/default/realtime"

BRONZE_PATH = f"{VOLUME_BASE}/delta/bronze/shipment_events"
SILVER_PATH = f"{VOLUME_BASE}/delta/silver/shipment_events"
BRONZE_CHECKPOINT = f"{VOLUME_BASE}/checkpoints/v3/bronze/shipment_events"
SILVER_CHECKPOINT = f"{VOLUME_BASE}/checkpoints/v3/silver/shipment_events"
```

Why the external ADLS container looked empty

After switching to /Volumes/..., Bronze/Silver/Gold were written to Databricks managed storage. They were no longer written to the standalone ADLS realtime container.

Issue 4 - Infinite Streaming Trigger Not Supported



Serverless rejected the indefinite trigger

Key point

The fix is a finite available-now micro-batch that drains currently available events and then terminates.

Fix Issue 4 - availableNow Trigger

Bronze

```
bronze_query = (
    bronze.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", BRONZE_CHECKPOINT)
    .trigger(availableNow=True)
    .start(BRONZE_PATH)
)
bronze_query.awaitTermination()
```

Silver

```
silver_query = (
    silver.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", SILVER_CHECKPOINT)
    .trigger(availableNow=True)
    .start(SILVER_PATH)
)
silver_query.awaitTermination()
```

Issue 5 - Events Sent but 0 Rows Processed

Two conditions contributed: the available-now run checked before the new events were sent, and checkpoint state could cause a later run to skip offsets already considered. The recovery used a fresh checkpoint version and earliest offsets for the deliberate test.

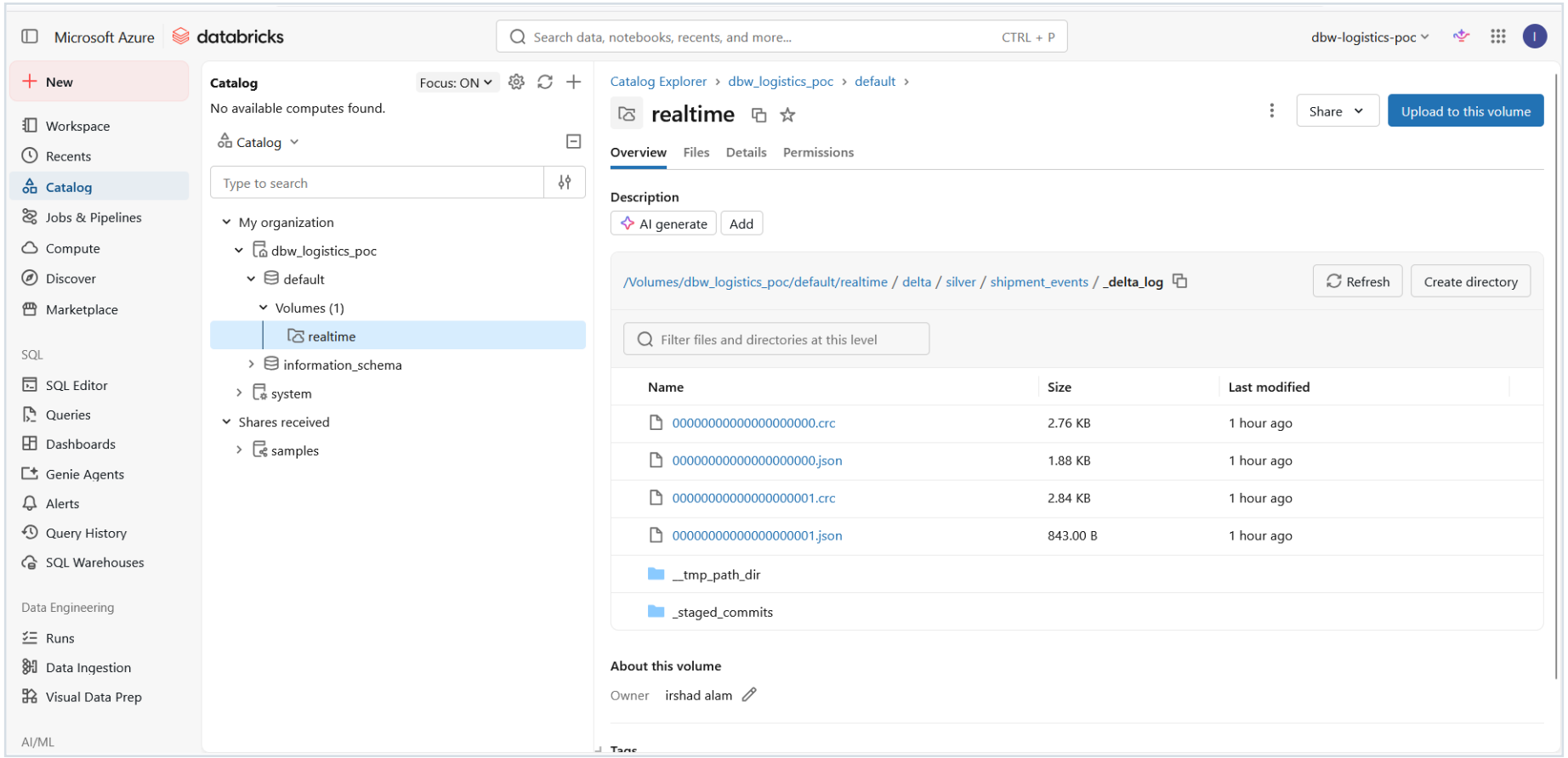
Recovery settings

```
BRONZE_CHECKPOINT = f"{VOLUME_BASE}/checkpoints/v3/bronze/shipment_events"
SILVER_CHECKPOINT = f"{VOLUME_BASE}/checkpoints/v3/silver/shipment_events"

kafka_options = {
    # ... other Kafka options ...
    "startingOffsets": "earliest",
    "failOnDataLoss": "false",
}
```

1. Send 50 events locally: python producer\send_events.py --count 50
2. Clear notebook state/output if needed.
3. Run stream_to_delta.py with the v3 checkpoint paths.
4. Inspect the query profile and Catalog Volume.

Bronze Ingestion - Successful Run

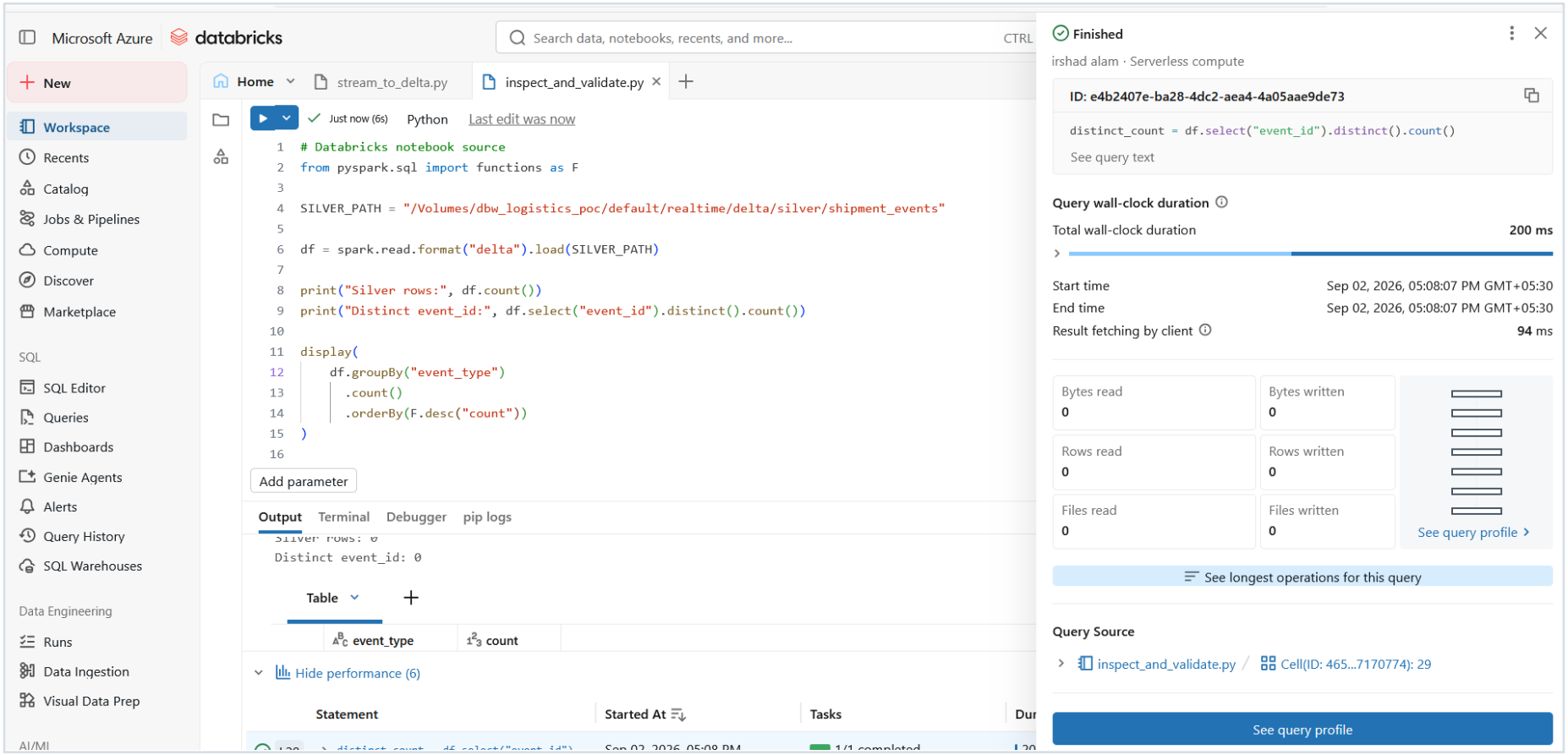


Serverless available-now run wrote 50 raw rows

Key point

This is the successful Event Hubs -> Databricks Bronze ingestion milestone.

Verify Data in Unity Catalog

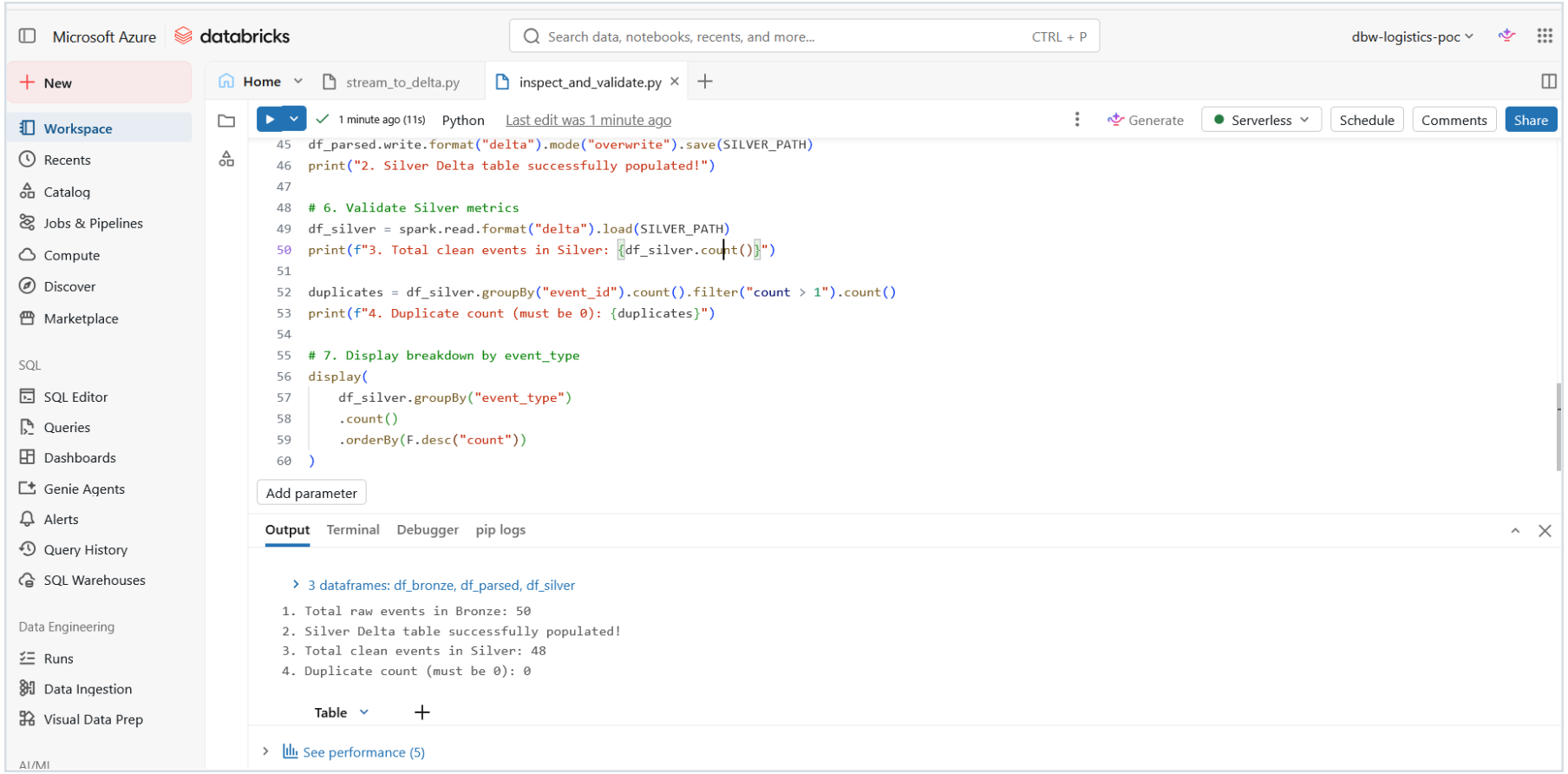


Delta files and _delta_log visible in the realtime Volume

Key point

Verify the actual storage location in Catalog Explorer. With /Volumes paths, do not expect the standalone ADLS container to contain these files.

Issue 6 - Initial Silver Validation Returned 0



Silver rows = 0 and distinct event_id = 0

Key point

Bronze ingestion had succeeded, but Silver was not yet populated in this validation attempt. The recovery reparsed Bronze and wrote Silver before validating it.

Silver Recovery - Parse Bronze

Schema and paths

```
from pyspark.sql import functions as F
from pyspark.sql.types import StructType, StructField, StringType, LongType, DoubleType

VOLUME_BASE = "/Volumes/dbw_logistics_poc/default/realtime"
BRONZE_PATH = f"{VOLUME_BASE}/delta/bronze/shipment_events"
SILVER_PATH = f"{VOLUME_BASE}/delta/silver/shipment_events"

event_schema = StructType([
    StructField("event_id", StringType(), False),
    StructField("order_id", LongType(), False),
    StructField("event_type", StringType(), False),
    StructField("event_ts", StringType(), False),
    StructField("region", StringType(), False),
    StructField("revenue", DoubleType(), True),
    StructField("fulfillment_minutes", LongType(), True),
    StructField("producer_seq", LongType(), True),
])
```

Silver Recovery - Deduplicate and Validate

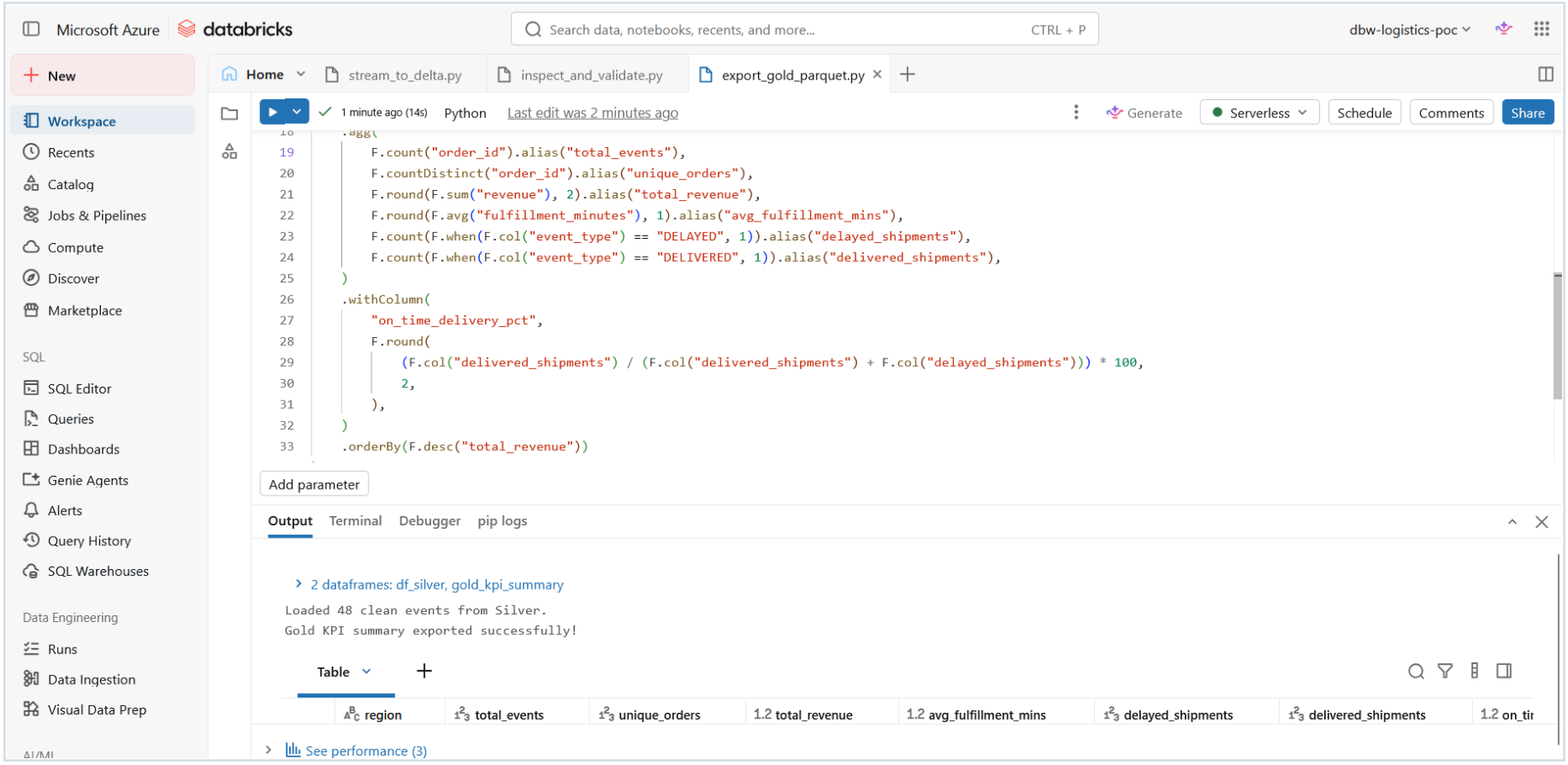
Recovery and validation

```
df_bronze = spark.read.format("delta").load(BRONZE_PATH)
print("1. Total raw events in Bronze:", df_bronze.count())

df_parsed = (
    df_bronze
    .withColumn("j", F.from_json("json_body", event_schema))
    .select("j.*", "partition", "offset", "eventhub_enqueued_ts", "ingested_at")
    .withColumn("event_ts", F.to_timestamp("event_ts"))
    .filter(F.col("event_id").isNotNull())
    .dropDuplicates(["event_id"])
)

df_parsed.write.format("delta").mode("overwrite").save(SILVER_PATH)
df_silver = spark.read.format("delta").load(SILVER_PATH)
print("3. Total clean events in Silver:", df_silver.count())
duplicates = df_silver.groupBy("event_id").count().filter("count > 1").count()
print("4. Duplicate count (must be 0):", duplicates)
```

Silver Validation - Successful Recovery



Bronze = 50, Silver = 48, duplicate count = 0

Key point

The producer deliberately injected duplicate event IDs. Deduplication reduced 50 raw events to 48 clean events while leaving zero duplicate event IDs in Silver.

Build the Gold KPI Summary

Aggregation

```
from pyspark.sql import functions as F

VOLUME_BASE = "/Volumes/dbw_logistics_poc/default/realtime"
SILVER_PATH = f"{VOLUME_BASE}/delta/silver/shipment_events"
GOLD_PATH = f"{VOLUME_BASE}/gold/shipment_summary"

df_silver = spark.read.format("delta").load(SILVER_PATH)

gold_kpi_summary = (
    df_silver.groupBy("region")
    .agg(
        F.count("order_id").alias("total_events"),
        F.countDistinct("order_id").alias("unique_orders"),
        F.round(F.sum("revenue"), 2).alias("total_revenue"),
        F.round(F.avg("fulfillment_minutes"), 1).alias("avg_fulfillment_mins"),
        F.count(F.when(F.col("event_type") == "DELAYED", 1)).alias("delayed_shipments"),
        F.count(F.when(F.col("event_type") == "DELIVERED", 1)).alias("delivered_shipments"),
    )
)
```

Build the Gold KPI Summary - Continued

Write Gold Parquet

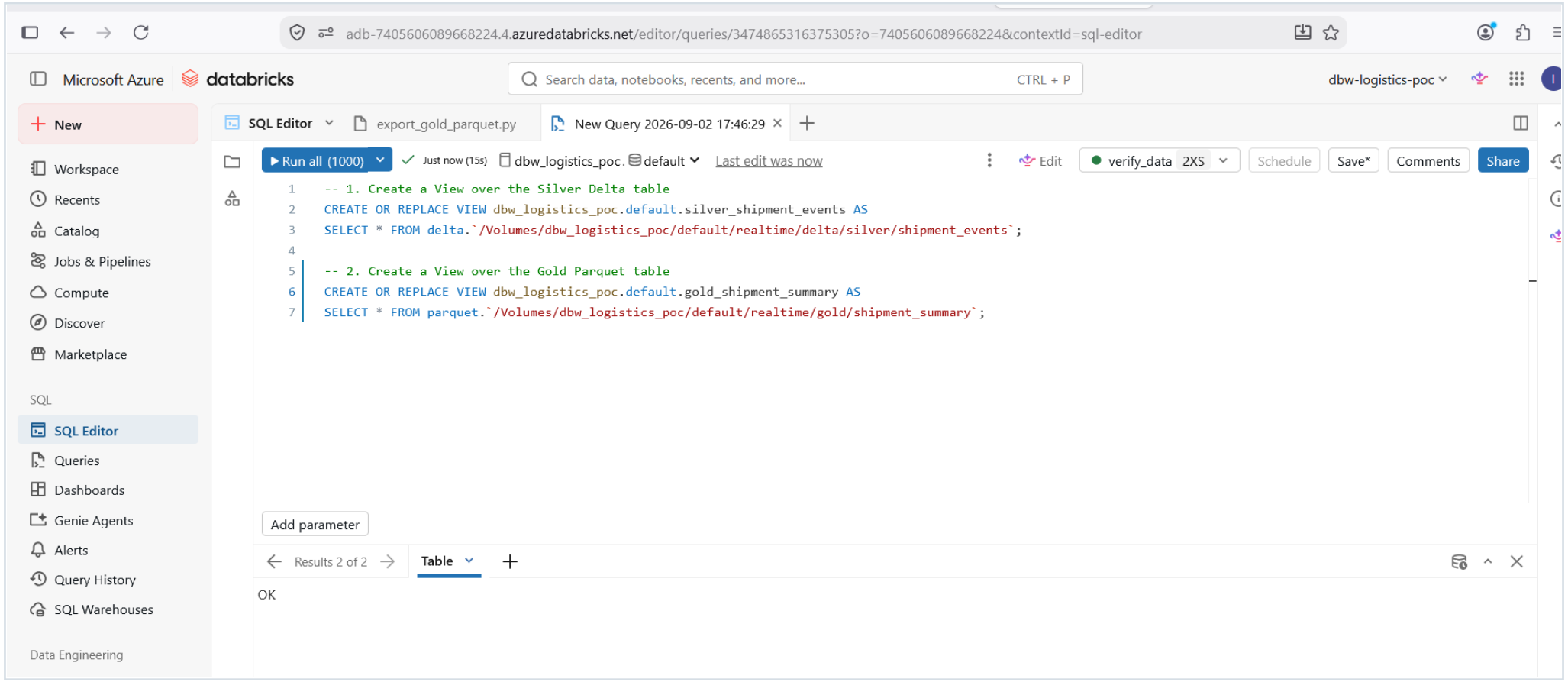
```
.withColumn(
    "on_time_delivery_pct",
    F.round(
        (F.col("delivered_shipments") /
         (F.col("delivered_shipments") + F.col("delayed_shipments"))) * 100,
        2,
    ),
)
.orderBy(F.desc("total_revenue"))

gold_kpi_summary.write.mode("overwrite").parquet(GOLD_PATH)
print("Gold KPI summary exported successfully!")
display(gold_kpi_summary)
```

Gold output

The curated Gold dataset contains regional order counts, revenue, average fulfillment time, delayed/delivered counts, and an on-time delivery percentage.

Gold KPI Export - Success

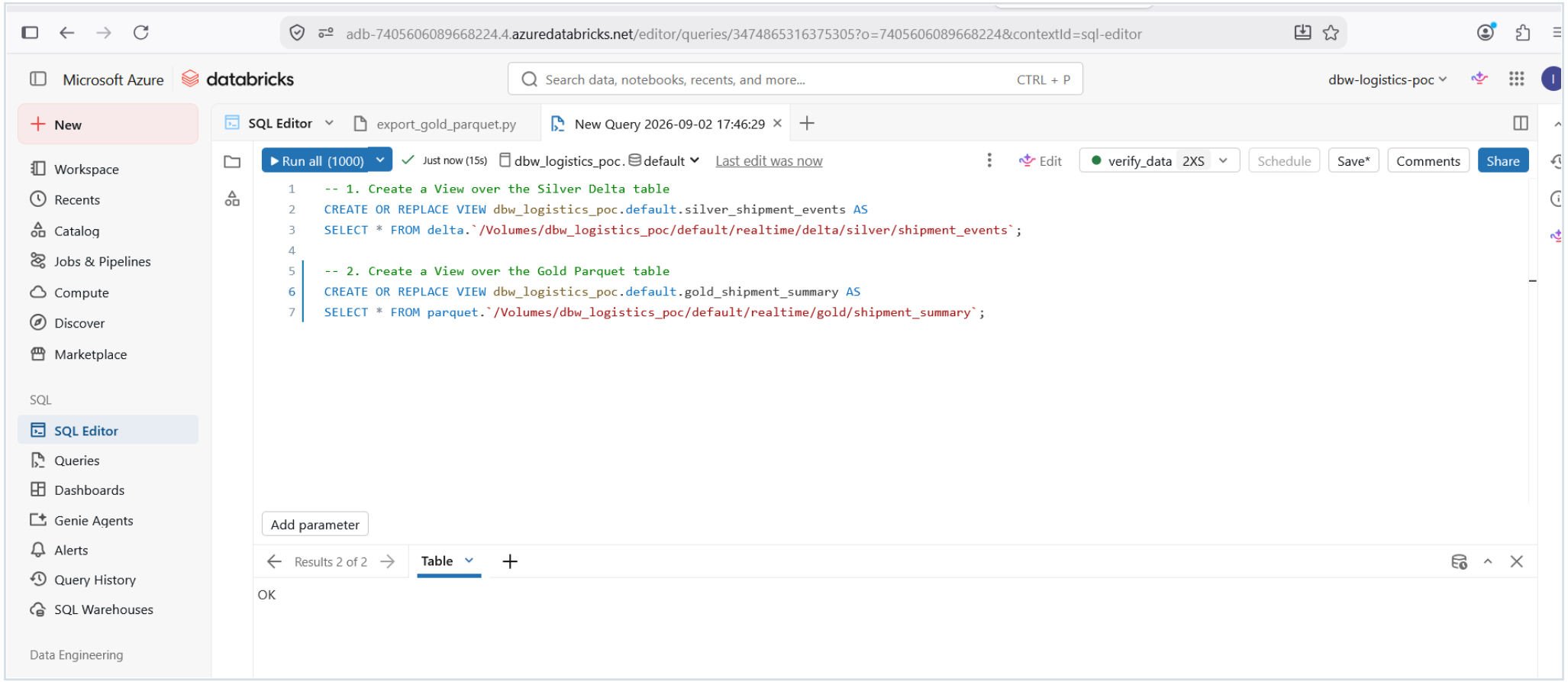


Gold KPI summary exported from the clean Silver layer

Key point

This completes the Medallion processing path: raw Bronze -> deduplicated Silver -> business-oriented Gold.

Databricks SQL Serving



Queryable SQL views over Silver Delta and Gold Parquet

Key point

Create views over the Volume paths so the curated data can be queried with standard SQL.

Create Databricks SQL Views

Databricks SQL

```
CREATE OR REPLACE VIEW dbw_logistics_poc.default.silver_shipment_events AS
SELECT * FROM delta.`/Volumes/dbw_logistics_poc/default/realtime/delta/silver/shipment_events`;

CREATE OR REPLACE VIEW dbw_logistics_poc.default.gold_shipment_summary AS
SELECT * FROM parquet.`/Volumes/dbw_logistics_poc/default/realtime/gold/shipment_summary`;
```

Why views are useful

They provide stable SQL names over the underlying Delta/Parquet paths, making verification and downstream analysis easier.

Verification Queries

Gold KPI verification

```
SELECT
    region,
    unique_orders,
    total_revenue,
    avg_fulfillment_mins,
    delayed_shipments,
    delivered_shipments,
    on_time_delivery_pct
FROM dbw_logistics_poc.default.gold_shipment_summary
ORDER BY total_revenue DESC;
```

Silver detailed verification

```
SELECT
    event_type,
    region,
    count(*) AS event_count,
    round(avg(revenue), 2) AS avg_revenue,
    min(event_ts) AS earliest_event,
    max(event_ts) AS latest_event
FROM dbw_logistics_poc.default.silver_shipment_events
GROUP BY event_type, region
ORDER BY event_count DESC;
```

Synapse Serverless SQL - Storage Requirement

Important boundary

Synapse cannot query a Databricks-managed /Volumes path directly. To use Synapse Serverless, write or copy the Gold Parquet dataset to ADLS (or another storage endpoint that Synapse can read).

Expected ADLS path pattern:

```
https://<storage-account>.dfs.core.windows.net/realtime/gold/shipment_summary/*.parquet
```

Schema alignment

The reference Synapse SQL expects total_orders, revenue, and delayed_shipments, while the final Databricks Gold logic uses unique_orders and total_revenue. Align the Parquet column names or update the SQL WITH clause before running Synapse.

Synapse Serverless SQL - Smoke Test and Narrow Query

A. Smoke test

```
SELECT TOP 100 *
FROM OPENROWSET(
    BULK 'https://<storage-account>.dfs.core.windows.net/realtime/gold/shipment_summary/*.parquet',
    FORMAT = 'PARQUET'
) AS rows;
```

B. Read only required columns

```
SELECT
    region,
    total_orders,
    delayed_shipments,
    revenue
FROM OPENROWSET(
    BULK 'https://<storage-account>.dfs.core.windows.net/realtime/gold/shipment_summary/*.parquet',
    FORMAT = 'PARQUET'
)
WITH (
    region VARCHAR(50),
    total_orders BIGINT,
    revenue FLOAT,
    delayed_shipments BIGINT
) AS gold
ORDER BY region;
```

Synapse Serverless SQL - Business Totals

C. Aggregate business totals

```
SELECT
    SUM(total_orders) AS total_orders,
    ROUND(SUM(revenue), 2) AS revenue,
    SUM(delayed_shipments) AS delayed_shipments
FROM OPENROWSET(
    BULK 'https://<storage-account>.dfs.core.windows.net/realtime/gold/shipment_summary/*.parquet',
    FORMAT = 'PARQUET'
)
WITH (
    total_orders BIGINT,
    revenue FLOAT,
    delayed_shipments BIGINT
) AS gold;
```

Cost lesson

For serverless lake queries, select only the columns and files you need. Inspect data processed/scanned in the query details.

Fabric Real-Time Intelligence - KQL Table

Implementation status

This is a reference path for future completion or repractice. It was not validated in the completed run.

KQL DDL

```
.create table ShipmentEvents (
    event_id:string,
    order_id:long,
    event_type:string,
    event_ts:datetime,
    region:string,
    revenue:real,
    fulfillment_minutes:long,
    producer_seq:long
)
```

Fabric KQL Verification Queries

Arrival, time bins, and delays

```
ShipmentEvents
| take 20

ShipmentEvents
| summarize events=count() by bin(event_ts, 5m), event_type
| order by event_ts desc

ShipmentEvents
| where event_type == "DELAYED"
| summarize delayed=count() by region
| order by delayed desc
```

Freshness and duplicate detection

```
ShipmentEvents
| summarize latest_event=max(event_ts)
| extend freshness_seconds = datetime_diff("second", now(), latest_event)

ShipmentEvents
| summarize copies=count() by event_id
| where copies > 1
| order by copies desc
```

Fabric Setup Sequence

1. Create a Fabric workspace on available Fabric capacity or trial.
2. Create an Eventhouse and KQL database.
3. Create ShipmentEvents using the KQL DDL.
4. Create an Eventstream and add Azure Event Hubs as the source.
5. Select shipment-events and configure the connection/consumer settings.
6. Add Eventhouse/KQL as the destination and map JSON fields.
7. Publish the Eventstream.
8. Send a new local batch with send_events.py.
9. Run KQL arrival, grouping, delay, duplicate, and freshness queries.
10. Create a small Lakehouse/shortcut and semantic model if available.

Monitoring Checklist

Platform health

- Event Hubs: Incoming Messages, Incoming Bytes, throttling, and errors.
- Databricks: rows read/written, query duration, checkpoint paths, and failures.
- Data freshness: compare max(event_ts) with current UTC time.

Data quality and serving

- Silver: row count, distinct event_id count, duplicate count.
- Gold: KPI totals by region and latest event timestamp.
- Fabric/Synapse when implemented: ingestion/query status and data processed.

Failure and Recovery Quick Reference

Failure	Working Recovery
Connection string malformed	Correct .env and copy Connection string-primary key
Databricks secret missing	Create secret; use only a temporary secure fallback if required
Serverless storage-key config blocked	Use Unity Catalog Volume or governed external access
Infinite stream unsupported	Use trigger(availableNow=True)
0 rows after send	Send batch before run; deliberate fresh checkpoint for recovery test
Silver rows = 0	Parse Bronze, deduplicate, write Silver, then validate

Cleanup and Cost Control

Azure CLI - delete the POC resource group

```
az group delete --name rg-logistics-poc --yes --no-wait
```

- Stop or suspend Databricks SQL warehouses after testing.
- Stop or terminate paid Databricks compute when applicable.
- Delete the POC resource group only after saving required outputs.
- Fabric resources may require separate cleanup.
- Rotate credentials that were exposed in notebook code, terminal history, screenshots, or repositories.

Fast Rerun Checklist

- ☐ Activate Python environment and install requirements.
- ☐ Confirm .env contains a valid Event Hubs connection string with no placeholders.
- ☐ Run send_events.py --count 20.
- ☐ Run receive_events.py and confirm events arrive.
- ☐ Open Databricks and verify the realtime Volume exists.
- ☐ For availableNow, send the main batch before starting the Databricks stream.
- ☐ Run stream_to_delta.py and confirm Bronze rows are written.
- ☐ Run inspect_and_validate.py; expect Silver > 0 and duplicate count = 0.
- ☐ Run export_gold_parquet.py and confirm Gold KPI output.
- ☐ Create or refresh SQL views and run Gold/Silver queries.
- ☐ For Synapse, ensure Gold Parquet is in ADLS rather than only /Volumes.
- ☐ For Fabric, publish Eventstream and run KQL verification.
- ☐ Capture results and stop compute.

Issue-to-Fix Reference - Part 1

Symptom	Root Cause	Fix
[Errno 11001] getaddrinfo failed	.env still has placeholder host	Replace placeholders with real Event Hubs values
Connection string blank/malformed	Wrong field, bad .env, or formatting	Copy Connection string-primary key; verify Python-read length
Secret does not exist	Secret scope/key not created	Create secret or temporary secure fallback
CONFIG_NOT_AVAILABLE	Serverless blocks storage-key Spark config	Use UC governed access/Volume or different compute

Issue-to-Fix Reference - Part 2

Symptom	Root Cause	Fix
JVM was not initialised	Secondary symptom after unsupported Spark config	Fix the primary CONFIG_NOT_AVAILABLE error
INFINITE_STREAMING_TRIGGER_NOT_SUPPORTED	Indefinite trigger not allowed	Use trigger(availableNow=True)
No rows after send_events.py	Timing and/or checkpoint state	Send events first; rerun with intended checkpoint
Silver rows = 0	Silver was not populated	Parse Bronze, deduplicate, write Silver, validate
Synapse cannot use /Volumes	Databricks Volume is not an ADLS URL	Write/copy Gold to ADLS and use OPENROWSET

Security and GitHub Safety Checklist

- Never commit .env.
- Never commit Event Hubs connection strings, storage keys, SAS tokens, client secrets, or Databricks tokens.
- Do not publish screenshots that show credentials.
- Use .env.example with placeholders only.
- Prefer managed identity, Unity Catalog credentials, or Databricks secret scopes over inline keys.
- Rotate any credential that appeared in a screenshot or pasted notebook cell.
- Before pushing to GitHub, search for Endpoint=sb://, SharedAccessKey=, AccountKey, SAS signatures, and long token strings.

Final Revision Summary

- Event Hubs producer and receiver: working.
- Databricks Serverless Bronze ingestion: 50 raw events.
- Silver recovery and deduplication: 48 clean events, 0 duplicate event IDs.
- Gold KPI export: working.
- Databricks SQL views: working.
- Serverless lesson: use availableNow for this interactive POC pattern.
- Storage lesson: /Volumes is Databricks managed storage; Synapse needs ADLS-accessible files.
- Fabric/Synapse/Purview: reference paths until separately implemented and verified.